

Pengembangan Aplikasi Mobile

Laporan Tugas Modul Praktikum 4–State Hoisting, Perubahan Konfigurasi, dan Side-effect

Penyusun

Divana Mutiara Zaquina

245150400111053

SI-D



Program Studi Sistem Informasi
Departemen Sistem Informasi
Fakultas Ilmu Komputer
Universitas Brawijaya

2026

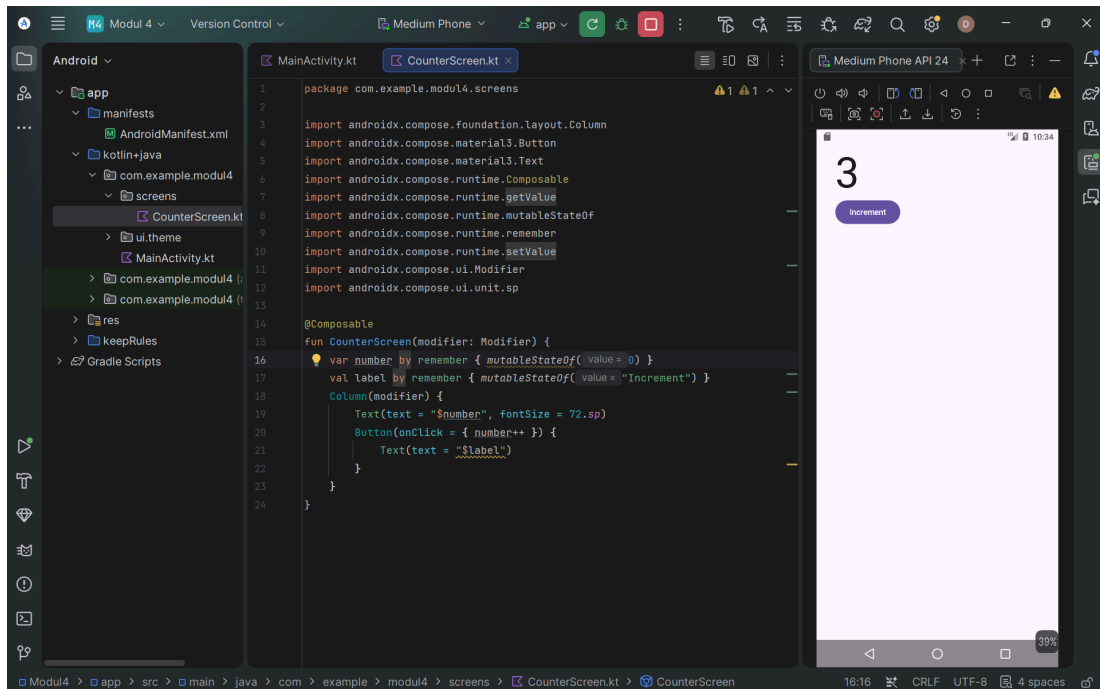
DAFTAR ISI

DAFTAR ISI.....	2
1.3. Latihan Praktik 1: Fungsi Setcontent Dan Activity Lifecycle.....	3
1.4. Latihan Praktik 2: Layouting & Preview.....	5
1.5. Tugas Praktik.....	9
Link Github.....	11

4.5 Percobaan

4.5.1. State Hosting

1. Menjalankan program pada emulator



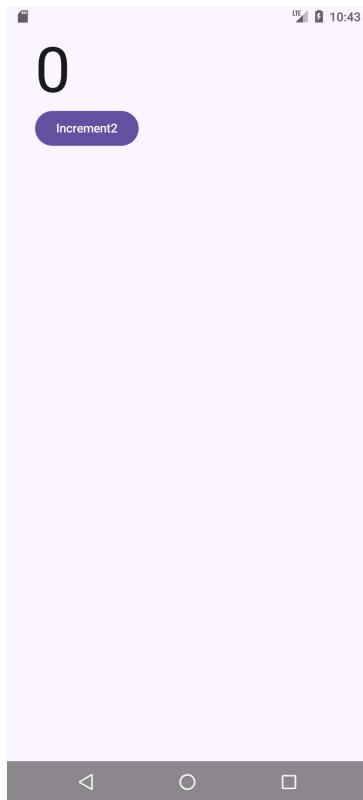
- a. Setiap kali tombol Increment diklik, nilai angka yang ditampilkan pada layar bertambah satu.
 - b. Dapat terjadi karena event onClick pada Button menjalankan operasi number++. Operasi tersebut menambah nilai state number sebesar satu. Ketika nilai state berubah, Jetpack Compose melakukan recomposition sehingga nilai baru ditampilkan pada Composable Text.
 - c. Variabel state yang berperan pada Composable Text adalah number.
 - d. Variabel state yang berperan adalah label.
2. Mengubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik)

- a. Perubahan dilakukan pada bagian event onClick Button dan nilai variabel label.

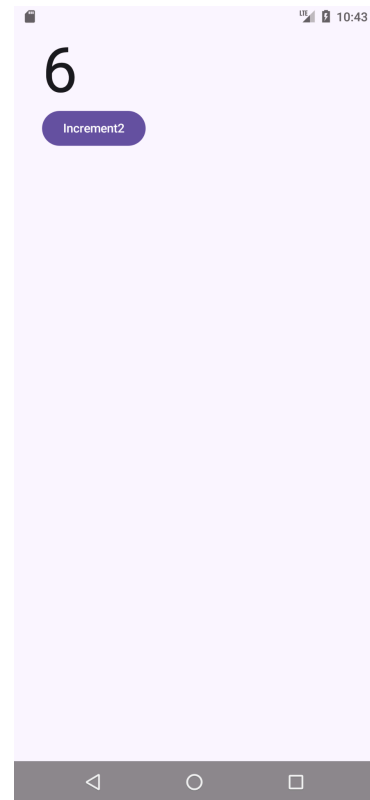
```
onButtonClick = { number++ } → onButtonClick = { number+=2 }
```

```
label = "Increment" → label = "Increment 2"
```

- b. Screenshot before vs after



vs



3. Melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen.
4. Menjalankan program
- a. Program tetap berjalan dengan baik dan menghasilkan tampilan serta perilaku yang sama.

- b. Proses increment dilakukan pada lambda onClick ketika CounterScreen dipanggil, yaitu pada bagian onClick = { number++ }.
- c. State number dideklarasikan pada fungsi onCreate(), CounterScreen menerima nilai tersebut melalui parameter number. Sedangkan label Button diberikan melalui parameter label. Jadi, CounterScreen hanya bertanggung jawab menampilkan data dan meneruskan event melalui callback.

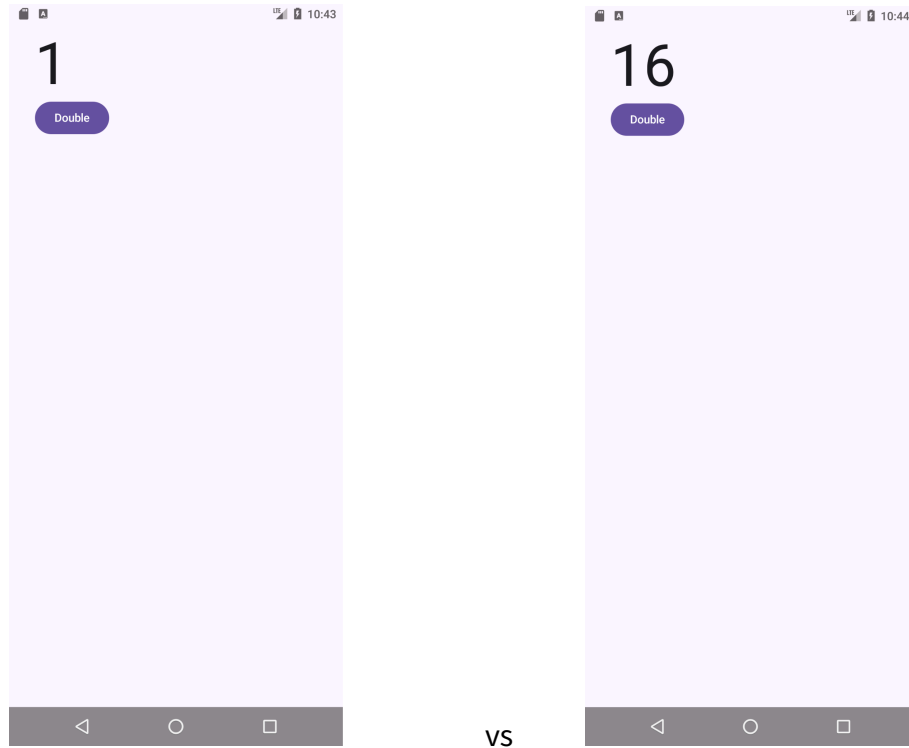
5. Perubahan kebutuhan fungsional aplikasi

Kode program:

```
var number by remember { mutableStateOf(1) }  
CounterScreen(Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number*=2 })
```

6. Menjalankan program

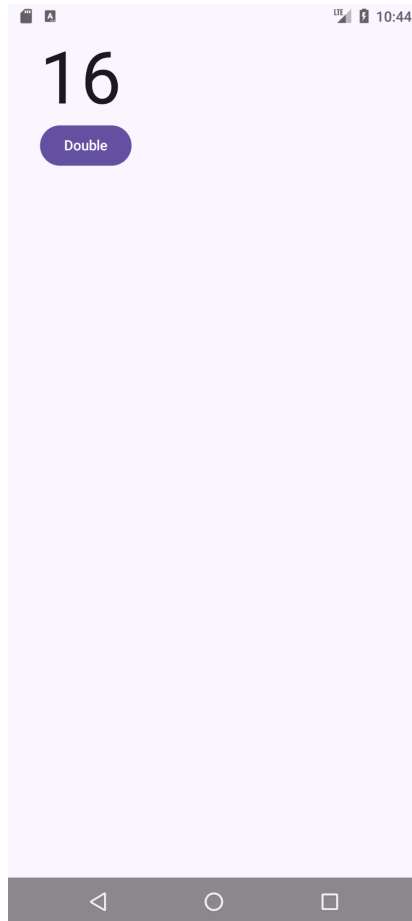
- a. Screenshot sebelum dan sesudah tombol dipencet beberapa kali



- b. Tidak ada perubahan pada fungsi Composable CounterScreen. Perubahan hanya dilakukan pada state dan lambda onClick ketika CounterScreen dipanggil di dalam onCreate()
7. State hoisting mempermudah perubahan kebutuhan fungsional karena logika pengelolaan state dipisahkan dari komponen UI. Jadi, komponen UI menjadi lebih reusable, mudah dipelihara, dan dapat digunakan kembali untuk kebutuhan yang berbeda.

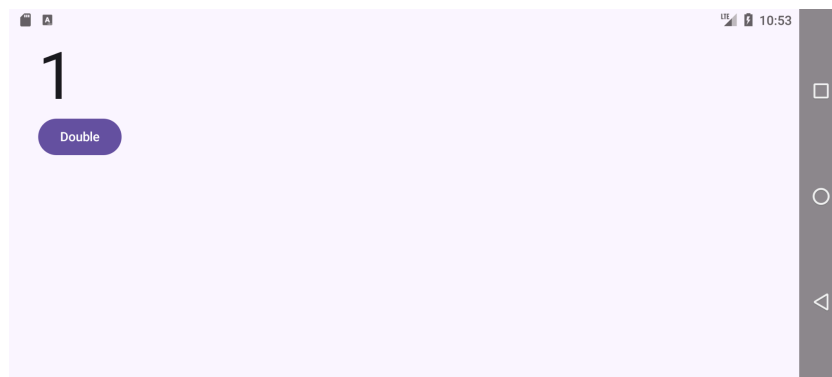
4.5.2 Perubahan Konfigurasi

1. Setelah tombol “Double” diklik beberapa kali:



2. Memutar posisi perangkat dari posisi portrait menjadi landscape

a. Screenshot:

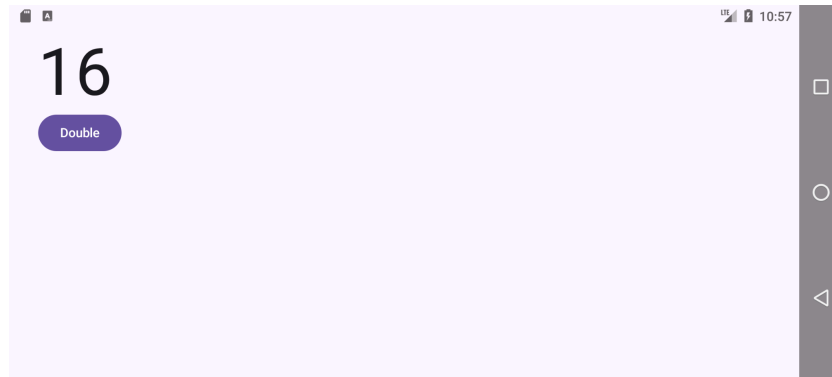


b. Angka yang semula 16 berubah kembali menjadi 1. Hal tersebut terjadi karena rotasi layar merupakan perubahan konfigurasi yang menyebabkan Activity

dibuat ulang. State kembali menggunakan nilai awal yang ditentukan pada deklarasi `mutableStateOf(1)` karena hanya disimpan menggunakan `remember`.

3. Mengubah kode program menjadi `rememberSaveable`

a. Screenshot:

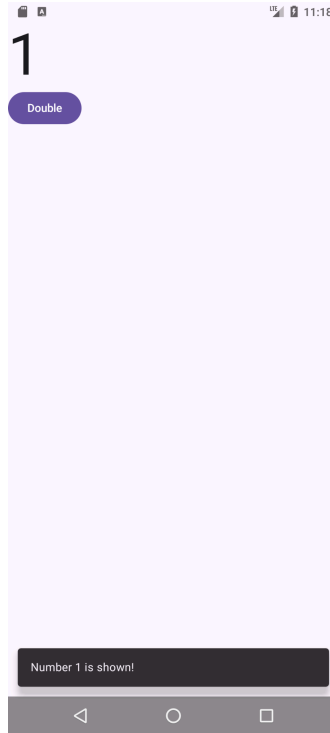


- b. Nilai yang ditampilkan tetap 16, yaitu nilai terakhir sebelum rotasi layar. Hal tersebut terjadi karena state number disimpan menggunakan `rememberSaveable`, sehingga nilainya dapat dipertahankan ketika terjadi perubahan konfigurasi seperti rotasi layar.
- c. `rememberSaveable` berfungsi untuk mempertahankan nilai state ketika terjadi perubahan konfigurasi yang menyebabkan Activity dibuat ulang, seperti rotasi layar. Tidak seperti `remember`, `rememberSaveable` menyimpan state melalui mekanisme `saved instance state` sehingga nilai dapat dipulihkan setelah Activity dibuat ulang.
4. Salah satu contoh penggunaan `rememberSaveable` adalah pada form pendaftaran atau form pengisian data pengguna. Misalnya, pengguna telah mengisi nama, alamat, atau data lainnya kemudian memutar perangkat dari portrait ke landscape. Dengan menggunakan `rememberSaveable`, nilai yang telah dimasukkan tidak berubah/hilang sehingga pengguna tidak perlu mengisi ulang data tersebut.

Manfaatnya adalah meningkatkan kenyamanan pengguna dan mencegah kehilangan input ketika terjadi perubahan konfigurasi perangkat.

4.5.3 LaunchedEffect

1. Menjalankan program



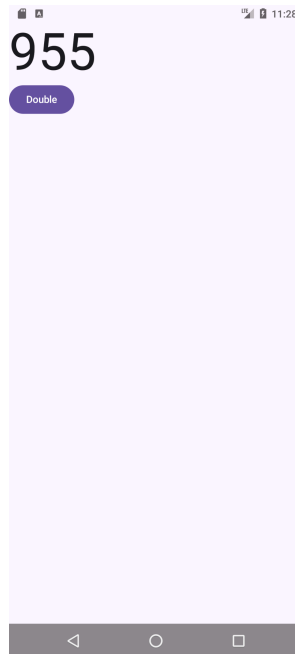
- a. Saat program pertama kali dijalankan, nilai number ditampilkan pada layar dan Snackbar menampilkan pesan "Number 1 is shown!"
 - b. Karena nilai number pada awalnya adalah 1, efek tersebut langsung dijalankan dan memanggil `showSnackbar()` untuk menampilkan pesan.
 - c. Snackbar dipicu oleh `LaunchedEffect(number)`. Ketika Composable pertama kali masuk ke dalam komposisi, efek dijalankan. Selain itu, setiap perubahan nilai number juga menyebabkan efek dijalankan kembali karena number digunakan sebagai key.
- #### 2. Meng-klik tombol “Double”

- a. Ketika tombol Double diklik, nilai number berubah dari 1 menjadi 2. Dan karena nilai number berubah, Snackbar menampilkan pesan baru “Number 2 is shown!”
- b. Karena number digunakan sebagai key pada LaunchedEffect(number), perubahan nilai tersebut menyebabkan efek dijalankan kembali.

Pra-Percobaan 2

1. Menjalankan program

- a. Pada saat pertama kali program dijalankan, muncul angka 1.
- b. Setelah menunggu, layar menampilkan angka random antara 100-999, yaitu 955.



- c. Pada awalnya, nilai number yang muncul adalah 1. Kemudian LaunchedEffect(Unit) dijalankan. Program menjalankan Repo.getData(), lalu menunggu: delay(2000). Setelah sekitar dua detik, Repo.getData()

mengembalikan angka acak antara 100–999 dan nilai tersebut diberikan kepada number.

2. Dari kedua percobaan

- a. Perbedaan utama terletak pada key yang digunakan untuk menentukan kapan efek dijalankan kembali. Pada `LaunchedEffect(state)`, efek bergantung pada nilai state yang digunakan sebagai key. Ketika nilai state tersebut berubah, efek akan dibatalkan dan dijalankan kembali dengan nilai terbaru. Sedangkan pada `LaunchedEffect(Unit)`, key menggunakan nilai tetap sehingga efek tidak dijalankan kembali hanya karena perubahan state lain.
- b. - `LaunchedEffect(number)` dijalankan saat pertama masuk komposisi dan ketika nilai number berubah.
- `LaunchedEffect(Unit)` dijalankan saat pertama kali masuk komposisi, tidak dipicu ulang oleh perubahan state lain.